École nationale de la statistique et de l'administration économique

STATAPP

Développement d'un agent conversationnel intelligent pour l'interrogation de bases de données en langage naturel

2025–2026

Membres de groupe :

Étudiant 1 : Vítor PRATA ZIELINSKI
Étudiant 2 : Bunrong BAN
Étudiant 3 : Montaha BEN ZAIED
Étudiant 4 : Senghak ROU

Encadrant :

Monsieur : Amadou DIOUF

1 Problème étudié

Relational databases are widely used to store structured information in many domains such as finance, commerce, and administration. These systems contain data that organizations rely on daily for analysis and decision making. However, accessing this information usually requires knowledge of **Structured Query Language (SQL)**, which is the language used to retrieve and manipulate data stored in database tables. This creates a gap between data availability and data usability, especially for business users who need quick analytical answers but do not have technical knowledge of database querying [3].

The **Text-to-SQL problem** aims to address this issue by translating natural language questions into executable SQL queries over a relational database. For example, a user may ask *"How many clients are in the database ?"*. The system then generates the corresponding SQL query, executes it on the database, and returns the result and response to the user in the structured format. Formally, given a natural language question x and a database schema s, the system must generate a SQL query y such that

$$f : \mathcal{X} \times \mathcal{S} \to \mathcal{Y}$$
$$(x, s) \mapsto y$$

where $\mathcal{X}$ represents the space of natural language questions (all possible user queries), $\mathcal{S}$ represents the space of database schemas (tables, columns, keys and relations), and $\mathcal{Y}$ represents the space of SQL queries. The generated query y must be syntactically valid and must also correctly reflect the analytical intent of the user.

Le Chatbot permet aux utilisateurs d'interroger une base de données en langage naturel (NL). Le système convertit les questions des utilisateurs en requêtes SQL. La base de données contient des tables telles que : **clients**, **dossiers**, and **transactions**.

2 Methodology

2.1 Pipeline Description

2.1.1 Architecture multi-agents

Les agents du système

Le projet contient cinq agents spécialisés :

Agent	Rôle
GuardrailsAgent	vérifie la sécurité des entrées et achemine les requêtes
SQLAgent	convertit le langage naturel en SQL
ErrorAgent	répare les requêtes SQL en cas d'échec
AnalysisAgent	génère des explications en langage naturel
VizAgent	crée des visualisations

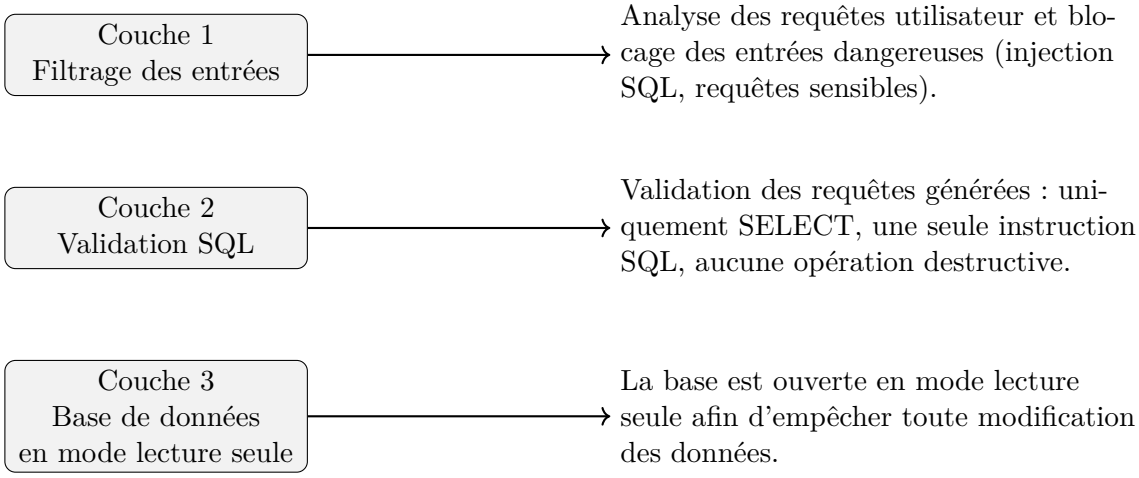
Nous avons opté pour une architecture multi-agents plutôt qu'un modèle unique pour plusieurs raisons :

- **Modularité** : chaque composant peut être développé et amélioré indépendamment par les membres de l'équipe.
- **Débogage facilité** : identifier et corriger les erreurs est plus simple quand les responsabilités sont séparées.
- **Sécurité renforcée** : GuardrailsAgent filtre les entrées dangereuses en amont.
- **Spécialisation** : chaque agent est optimisé pour sa tâche spécifique (SQL, visualisation, explication...).

Conclusion : Cette approche nous permet de répartir le travail efficacement et d'assurer une meilleure maintenabilité du système.

2.1.2 Architecture de sécurité

Afin de garantir une utilisation sécurisée du projet, nous avons mis en place une architecture de sécurité basée sur le principe de *defense-in-depth*. Cette approche repose sur plusieurs niveaux de protection complémentaires afin de limiter les risques à différentes étapes de l'interaction avec le système.



2.2 Processing Pipeline

The pipeline proceeds through the following stages **(Figure 1)**. Let Q denote the user question, db the SQLite database, and $S = \text{get_schema_text}(db)$ the extracted schema.

(1) Guardrails. The question is first evaluated by a guardrails layer :

$$Q \xrightarrow{\text{guardrails}} G, \quad G.\text{status} \in \{\text{OUT_OF_SCOPE},\ \text{NEEDS_CLARIFICATION},\ \text{READY_FOR_SQL}\}$$

The pipeline halts if the question is out of scope, returns a clarification request if incomplete, and proceeds only when $G.\text{status} = \text{READY_FOR_SQL}$.

(2) SQL generation. A candidate query is produced :

$$Y = \text{SQLAgent.generate_sql}(Q, S)$$

(3) Validation. The candidate is checked for safety and correctness :

$$\text{Valid}(Y) = \begin{cases} \text{true} & \text{if } Y \text{ is a safe, single } \texttt{SELECT} \text{ query with no PII columns} \\ \text{false} & \text{otherwise} \end{cases}$$

(4) Repair loop. If validation fails or execution raises an error, the system iterates :

$$Y_{k+1} = \text{ErrorAgent.repair_sql}(Q, S, Y_k, \varepsilon_k), \quad k \leq K_{\max} = 3$$

where ε_k is the error message at attempt k.

(5) Execution. Once valid, the query is executed : $R = (C, D)$, where C are the returned columns and D the rows.

(6) Response generation. The result is formatted and passed to the analysis agent :

$$A = \text{AnalysisAgent.summarize}(Q, Y, C, D), \qquad V = \text{VizAgent.generate}(Q, C, D)$$

producing a natural-language answer A and an optional visualization V. The full pipeline can thus be summarised as :

$$Q \longrightarrow G \longrightarrow Y \xrightarrow{\text{validate/repair}} R \longrightarrow (A, V)$$

The detailed algorithm and an illustrative example are provided in Appendix A.2.

2.3 Benchmark Structure

From now on, we used the Ollama LLM as the core of our chatbot, more specifically the llama3.2 model.

2.3.1 Query complexity

We used 100 questions/prompts and the expected SQL code for each one. For making our test more precise, we divided the prompts in levels of difficulty (1 being the easiest and 4 the hardest).

To better understand the meaning of the benchmark difficulty levels, we extract structural features from the reference SQL queries. These features include the average query length in tokens, as well as the proportion of queries containing advanced SQL constructs such as `JOIN`, `GROUP BY`, subqueries, Common Table Expressions (CTE), and window functions.

Table 3 shows that query complexity increases consistently across levels. Level 1 queries are short and involve only simple operations, while level 2 introduces joins and aggregations. Moreover, Level 3 queries frequently contain nested queries and CTEs, and Level 4 queries exist with the highest complexity, including analytical operations such as window functions and other operations (**Figure 4**).

2.4 Main Evaluation Metrics

The primary metric is **Execution Accuracy (EX)** [6, 5]. Given a reference SQL s_i^* and a generated SQL $\hat{s}_i$, both are executed against the database yielding result sets V_i^* and $\hat{V}_i$. EX is defined as :

$$\text{EX} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}(V_i^*, \hat{V}_i), \qquad \mathbf{1}(V_i^*, \hat{V}_i) = \begin{cases} 1 & \text{if } V_i^* = \hat{V}_i \\ 0 & \text{otherwise} \end{cases}$$

EX is preferred over Exact Match (EM), which compares SQL strings directly, because two syntactically different queries can produce identical results.

2.4.1 Jaccard Similarity

For queries where the generated SQL executes successfully but produces an incorrect result, we compute a continuous similarity score using the **Jaccard index** between the two result sets, treated as sets of rows :

$$J(V^*, \hat{V}) = \frac{|V^* \cap \hat{V}|}{|V^* \cup \hat{V}|}$$

2.4.2 Wilson Confidence Intervals

Since EX is a proportion estimated from a finite sample, we report 95% confidence intervals using the **Wilson score interval** [4] rather than the standard Wald interval. The Wilson interval is defined as :

$$\left(\frac{\hat{p} + \frac{z^2}{2n} \mp z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \right)$$

where $\hat{p}$ is the observed accuracy, n is the sample size, and $z = 1.96$ for a 95% confidence level. The Wilson interval has better coverage properties than the Wald interval when $\hat{p}$ is close to 0 or 1 [2], which is the case here given the low overall accuracy of 16%.

3 Results

3.1 Overall Performance

The model achieved an Execution Accuracy (EX) of 16% (16 out of 100 queries) with a 95% Wilson confidence interval of [10.1%, 24.4%]. This means that in roughly 1 in 6 questions, the generated SQL produced exactly the correct result when executed against the database.

Performance decreases sharply with increasing query complexity, dropping from 40% on simple queries to 0% on expert-level queries. A detailed breakdown of execution accuracy by difficulty level is reported in Table 1 .

3.2 Independence from Difficulty Level

Let p_ℓ denote the true success probability at level ℓ. We test $H_0 : p_1 = p_2 = p_3 = p_4$ against $H_1 : \exists\, \ell \neq \ell'$ such that $p_\ell \neq p_{\ell'}$, using a **Pearson chi-squared test of independence** on the contingency table of difficulty level versus binary outcome. The test statistic is :

$$\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}, \quad E_{ij} = \frac{R_i C_j}{n}$$

which under H_0 follows a χ^2 distribution with $(r-1)(c-1) = 3$ degrees of freedom. The test yields $\chi^2(3) = 15.31$, $p = 0.0016$, leading us to reject H_0 at the 5% significance level. There is strong statistical evidence that accuracy is **not independent of difficulty level** — the model's performance degrades systematically as query complexity increases.

4 Error Analysis

Following the taxonomy proposed by [6], we classify all incorrect queries into two high-level categories : **System Errors**, where the generated SQL is syntactically invalid and cannot be executed, and **Result Errors**, where the SQL executes successfully but produces an incorrect result.

4.1 High-Level Error Distribution

Of the 84 incorrect queries, 18 (21.4%) were System Errors and 66 (78.6%) were Result Errors, indicating that the model rarely produces outright invalid SQL. The dominant failure mode is therefore semantic rather than syntactic. A detailed breakdown of the error distribution across the 100 evaluated queries is provided in **Table 2**.

4.2 System Error Subcategories

System Errors are further broken down by the error message returned by SQLite (see **Table 3**). The dominance of *no such column* errors (77.8% of System Errors) suggests the model occasionally hallucinates column names that do not exist in the schema, a known failure mode in Text-to-SQL systems [6].

4.3 Result Error Subcategories

Result Errors are classified hierarchically following [6] : table selection is evaluated first, then column selection, JOIN correctness, condition filtering, and finally data processing (see **Table 4**).

Column-related errors (incorrect, missing, or extra columns) account for 57.6% of all Result Errors, making column selection the primary bottleneck of the model. Condition and filter errors represent a secondary failure mode at 19.7%.

4.4 Error Distribution by Difficulty Level

Error types vary systematically with query complexity (see **Table 5**). A notable finding is that **Level 1 queries produced zero System Errors** : all failures at this level were semantic. System Errors emerge only at Level 2 and above, increasing from 7 at Level 2 to 6 at Level 4. This suggests that syntactic failures are associated with complex SQL constructs such as CTEs, window functions, and multi-table joins, which appear only in higher difficulty levels.

5 What Makes a Query Hard ? Logistic Regression Analysis

5.1 Preliminary Analysis : Point-Biserial Correlations

Before fitting a regression model, we examine the bivariate association between each structural feature of the expected SQL and the binary outcome (correct / incorrect) using the point-biserial correlation coefficient, which is equivalent to Pearson's r when one variable is binary (see **Table 6**).

All significant correlations are negative, confirming that structural complexity is consistently associated with lower accuracy. Difficulty level is the strongest predictor ($r = -0.347$, $p < 0.001$), followed by query length and the presence of JOIN operations.

5.2 Perfect Separation

A notable finding emerges from the mean feature values stratified by outcome (see **Table 7**). The model **never** produced a correct result for any query involving a JOIN, a window function, or a CASE WHEN expression.

This constitutes a case of *perfect separation* [1] : these features perfectly predict failure in the observed sample. While this precludes stable maximum likelihood estimation for these variables, it is itself a substantive finding — the model systematically fails on relational and analytical SQL constructs.

5.3 Logistic Regression Model

Given the perfect separation issue, we fit a logistic regression model with an L_2 regularization penalty (ridge logistic regression) with $\lambda = 0.5$, which stabilizes the estimates while preserving interpretability. The model specification is

$$\log \frac{p_i}{1 - p_i} = \beta_0 + \beta_1 \cdot \text{Nivel}_i + \beta_2 \cdot \text{JOIN}_i + \beta_3 \cdot \text{CTE}_i + \beta_4 \cdot \text{WindowFunc}_i + \beta_5 \cdot \text{Tokens}_i + \varepsilon_i$$

where $p_i = P(\text{Correct}_i = 1)$ and all continuous predictors are standardized to mean 0 and standard deviation 1. Full coefficient estimates are reported in **Table 8** (Appendix).

5.4 Interpretation

The negative intercept ($\beta_0 = -2.467$) reflects the baseline low probability of success. Difficulty level has the largest statistically marginal effect : each unit increase in level multiplies the odds of success by $e^{-0.713} = 0.49$, roughly halving them. Query length also has a negative effect (OR = 0.354), consistent with longer queries being harder to translate correctly.

The positive coefficient for CTE ($\beta = +0.690$, OR = 1.99) is counterintuitive at first glance. However, it reflects a confounding effect : in this dataset, CTEs appear predominantly at Levels 3 and 4 alongside window functions — the true source of failure. With $n = 100$, the model lacks the statistical power to disentangle these collinear features.

These results should be interpreted with caution given the small sample size and the perfect separation issue. A larger benchmark dataset would be required to obtain stable, fully interpretable estimates.

6 Conclusion

These results suggest three concrete directions for improvement. First, the prompt template could be enriched with few-shot examples specifically targeting JOIN and multi-table queries, which represent the primary bottleneck. Second, a self-debugging loop — as proposed by [6] — could be integrated into the pipeline : given that the model already generates syntactically valid SQL in 82% of cases, passing the error message back to the model for one correction round is likely to yield meaningful gains at low cost. Third, the benchmark itself could be extended to compare multiple models under identical conditions, enabling a McNemar test to assess whether performance differences are statistically significant rather than attributable to sampling variability.

From a statistical standpoint, the small sample size ($n = 100$) limits the precision of the estimates — particularly the wide confidence intervals at Levels 3 and 4 — and prevents stable maximum likelihood estimation in the logistic regression due to perfect separation. A larger benchmark would be required to obtain fully reliable coefficient estimates and to power a formal comparison between models.

Références

[1] Adelin Albert and John A. Anderson. On the existence of maximum likelihood estimates in logistic regression models. *Biometrika*, 71(1) :1–10, 1984.

[2] Lawrence D. Brown, T. Tony Cai, and Anirban DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2) :101–133, 2001.

[3] Samuel Ojuri, The Anh Han, Raymond Chiong, and Alessandro Di Stefano. Optimizing text-to-sql conversion techniques through the integration of intelligent agents and large language models. *Information Processing and Management*, 62 :104136, 2025.

[4] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22 :209–212, 1927.

[5] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider : A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of EMNLP*, 2018.

[6] Bin Zhang, Yuxiao Ye, Guoqing Du, Xiaoru Hu, Zhishuai Li, Sun Yang, Chi Harold Liu, Rui Zhao, Ziyue Li, and Hangyu Mao. Benchmarking the text-to-sql capability of large language models : A comprehensive evaluation. *arXiv preprint arXiv :2403.02951*, 2024.

A Annexe

A.1 Pipeline de traitement

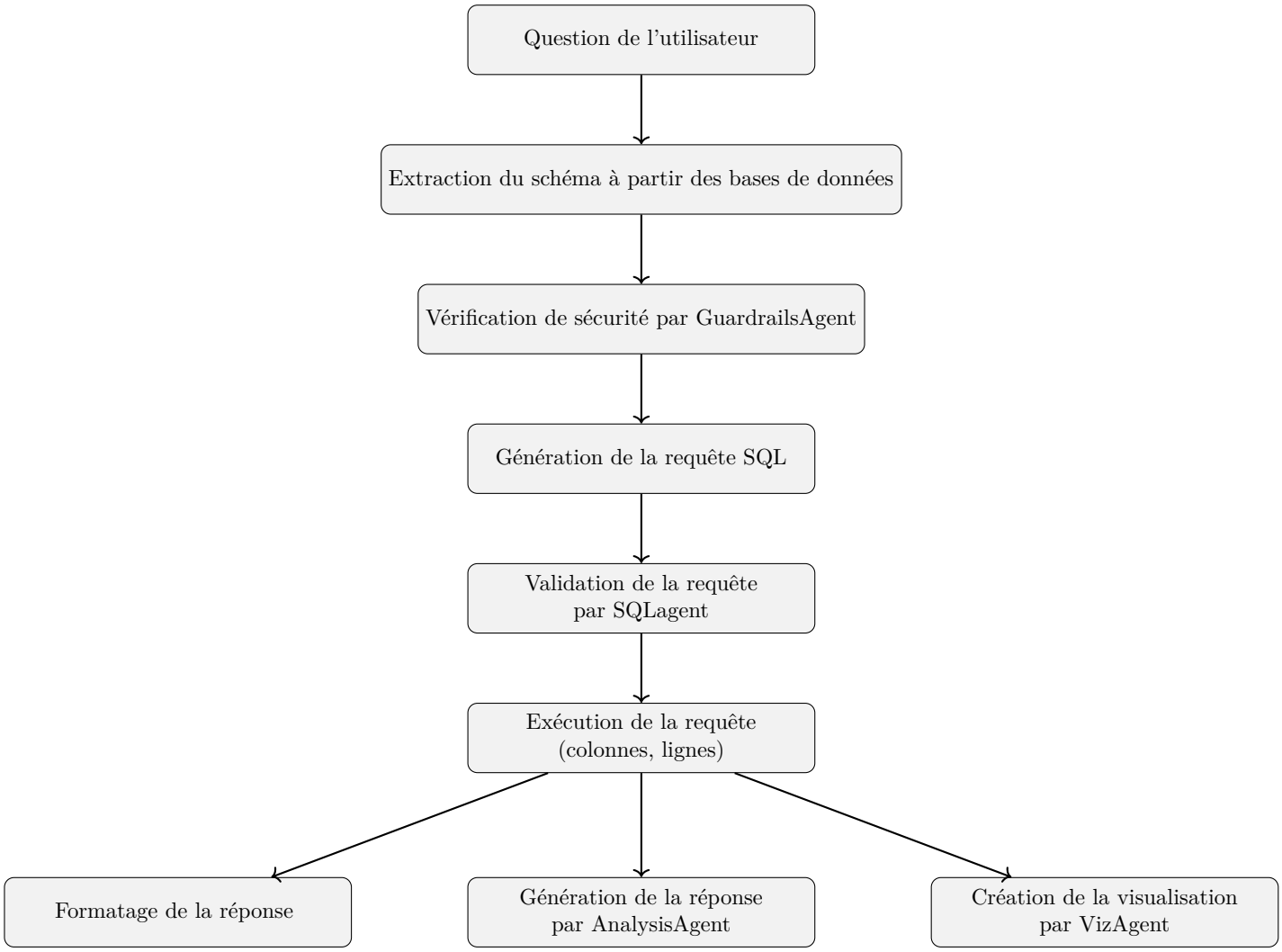


FIGURE 1 – Pipeline de traitement du système Text-to-SQL.

A.2 Algorithme du pipeline

```
Algorithm StatAppPipeline(db, Q)

1. S ← get_schema_text(db)
2. G ← GuardrailsAgent.evaluate(Q)

3. if G.status = OUT OF SCOPE:
       return refusal response
4. else if G.status = NEEDS CLARIFICATION:
       return clarification response
5. else:
       Y ← SQLAgent.generate_sql(Q, S)          // LLM-based

6. for k = 1 to Kmax + 1:
       if not Valid(Y):
            err ← validation error
       else:
            R ← execute_sql(db, Y)
            if R is successful:
                break
            err ← execution error

       if k = Kmax + 1:
            return error response

       Y ← ErrorAgent.repair_sql(Q, S, Y, err)          // LLM-based

7. Let R = (C, D)
8. F ← format_response_dict(C, D)
9. A ← AnalysisAgent.summarize(Q, Y, C, D, F.text)          // LLM-based
10. V ← VizAgent.generate(Q, C, D, infer_plotly(Q, C, D))       // LLM-based
11. return final structured response
```

This algorithm shows that the pipeline is not a single LLM call. Instead, it is a controlled sequence of stages : safety checking, SQL generation, validation, execution, repair, answer generation, and visualization. This modular structure improves reliability because each stage can be verified before moving to the next one.

A.3 Exemple d'exécution

Consider the question : *How many clients are there by segment client ?*

In this case, the system first extracts the schema from the database and applies the guardrails stage. Since the request is a valid analytical question, it is accepted for SQL generation.

The SQL agent may generate the following query :

```sql
SELECT clients.segment_client, COUNT(DISTINCT clients.client_id) AS nombre_clients
FROM clients
GROUP BY clients.segment_client
ORDER BY nombre_clients DESC, clients.segment_client
LIMIT 200;
```

This query is validated and then executed on the SQLite database. The result can be represented as :

$$R = \{(\text{commune}_1, c_1), (\text{commune}_2, c_2), \ldots, (\text{commune}_n, c_n)\}$$

The analysis agent then turns this result into a short natural-language explanation, while the visualization stage generates a bar chart because the result has two columns. The final response therefore contains the SQL query, the returned rows, the textual answer, and the visualization.

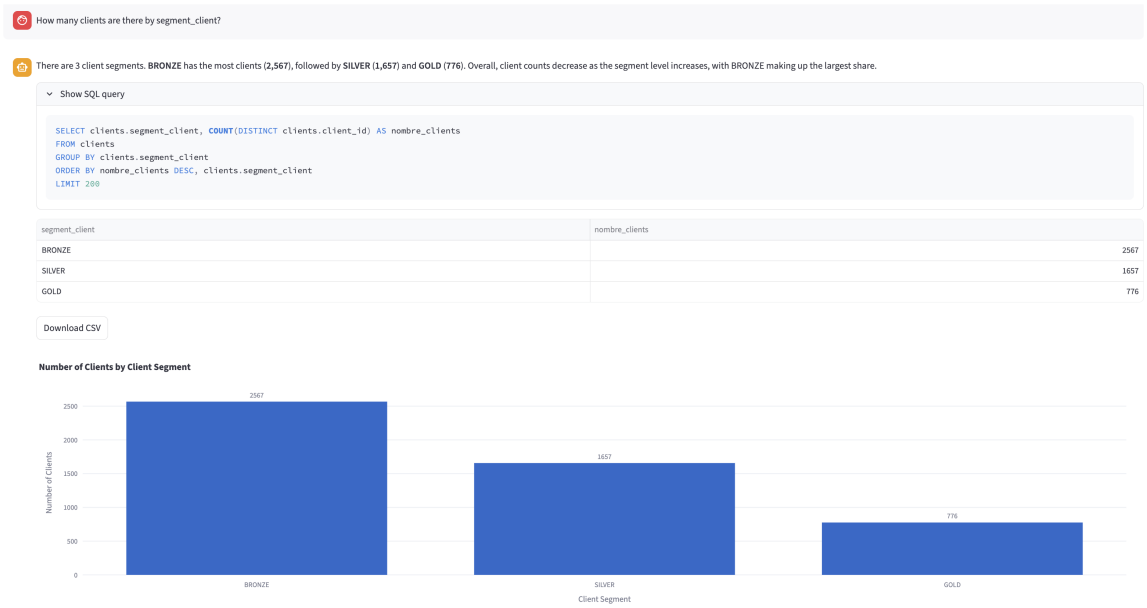


FIGURE 2 – Exemple de sortie de l'application.

This example illustrates the normal path of the pipeline in a simple case. In more difficult cases, the same process can also activate the repair loop before execution succeeds.

A.4 Query complexity

Level	Mean tokens	JOIN (%)	GROUP BY (%)	Subquery (%)	CTE (%)	Window (%)
1	8.20	0.00	0.00	0.00	0.00	0.00
2	16.23	31.43	57.14	0.00	0.00	0.00
3	25.12	32.00	52.00	68.00	36.00	0.00
4	48.20	53.33	46.67	53.33	53.33	53.33

FIGURE 3 – Structural complexity of SQL queries across difficulty levels.

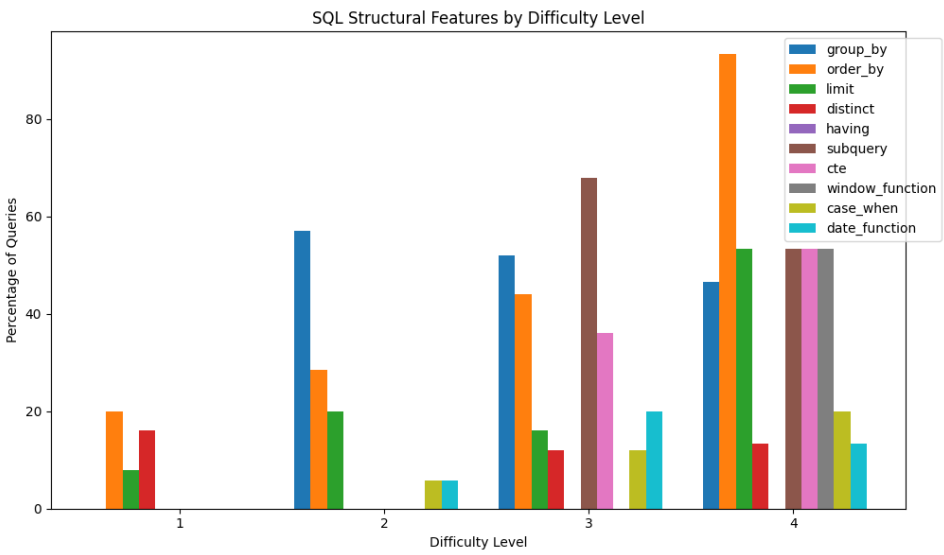


FIGURE 4 – Distribution of SQL structural features across difficulty levels.

A.5 Overall performance

Level	n	EX	95% CI
1 — Simple	25	40%	[23.4%, 59.3%]
2 — Intermediate	35	11.4%	[4.5%, 26.0%]
3 — Advanced	35	8%	[2.2%, 25.0%]
4 — Expert	15	0%	[0.0%, 20.4%]

Table 1 – Execution accuracy (EX) by difficulty level with Wilson 95% confidence intervals.

A.6 High level Error Distribution

Error Type	Count	%
Correct	16	16.0%
Result Error	66	66.0%
System Error	18	18.0%

Table 2 – High-level error distribution across 100 queries.

A.7 System Error subcategories

Subcategory	Count	%
No such column	14	77.8%
Syntax error	3	16.7%
No such function	1	5.6%

Table 3 – System Error subcategories.

A.8 Result Error Subcategories

Subcategory	Count	%
Incorrect column	23	34.8%
Condition/filter error	13	19.7%
Missing column	10	15.2%
Extra column	5	7.6%
Processing error	5	7.6%
Extra table	4	6.1%
Missing table	3	4.5%
JOIN error	2	3.0%
Incorrect table	1	1.5%

Table 4 – Result Error subcategories.

A.9 Error type distribution by difficulty level

Level	n	Correct	Result Error	System Error
1	25	10	15	0
2	35	4	24	7
3	25	2	18	5
4	15	0	9	6

Table 5 – Error type distribution by difficulty level.

A.10 Preliminary Analysis : Point-Biserial Correlations

Feature	r	p**-value**
Difficulty level	-0.347	0.0004 ***
Query length (tokens)	-0.267	0.0073 **
Contains JOIN	-0.265	0.0076 **
Contains GROUP BY	-0.245	0.0140 *
Contains subquery	-0.189	0.0597 .
Contains window function	-0.129	0.2019
Contains CTE	-0.125	0.2157
Contains date function	-0.088	0.3861

Significance : *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, . $p < 0.1$

Table 6 – Point-biserial correlations between SQL features and query outcome.

A.11 Perfect Separation

Feature	**Incorrect queries**	**Correct queries**
Contains JOIN	0.321	0.000
Contains window function	0.095	0.000
Contains CASE WHEN	0.095	0.000
Query length (tokens)	23.3	10.2
Difficulty level	2.45	1.50

Table 7 – Mean feature values by outcome.

A.12 Logistics Regression Model

Variable	**Coef.**	**OR**	**SE**	z	p
Intercept	-2.467	0.085	0.528	-4.674	< 0.001 ***
Difficulty level	-0.713	0.490	0.430	-1.657	0.097 .
JOIN	-0.883	0.414	0.614	-1.438	0.150
CTE	$+0.690$	1.994	0.514	$+1.342$	0.180
Window function	-0.256	0.774	0.731	-0.351	0.726
Query length	-1.039	0.354	0.780	-1.333	0.183

In-sample accuracy : 84.0%. Significance : *** $p < 0.001$, . $p < 0.1$

Table 8 – Ridge logistic regression results ($\lambda = 0.5$).